

计算理论与算法分析设计

教材: 张春霞

[1][王] 王晓东, 计算机算法设计与分析(第5/6版), 电子工业出版社.

[2][S] 段磊、唐常杰等译, Sipser著, 计算理论导引, 机械工业出版社.

参考资料:

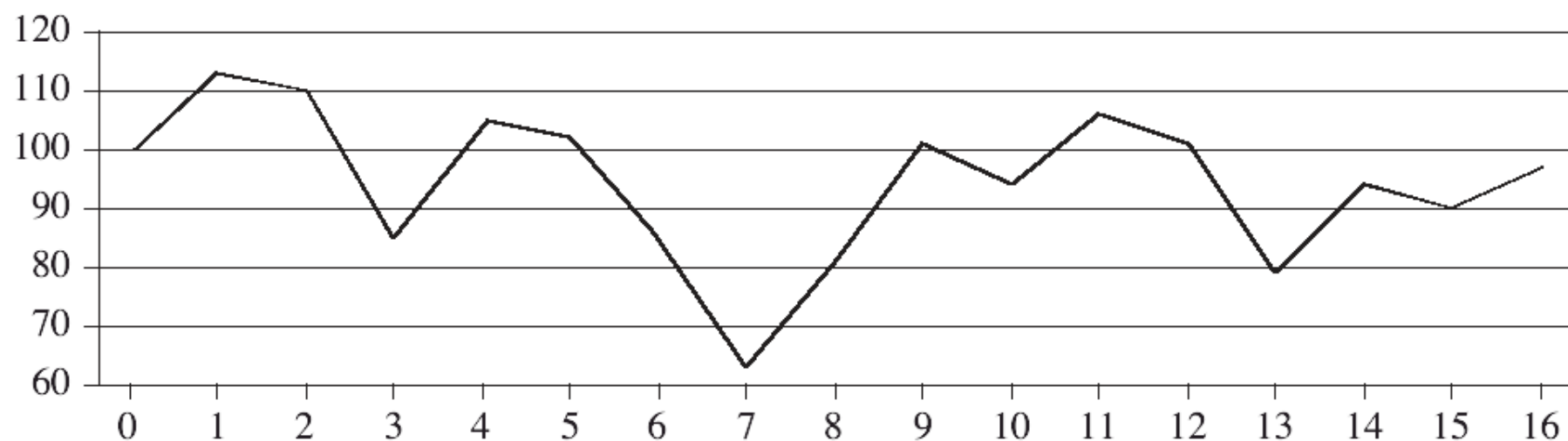
[3][C] 潘金贵等译, Cormen等著, 算法导论, 机械工业出版社.

[4][M] 黄林鹏等译, Manber著, 算法引论-一种创造性方法, 电子工业出版社.

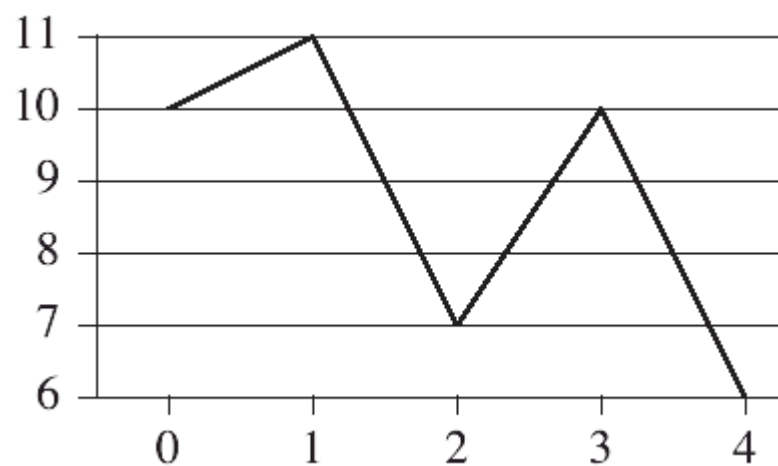
[5][L] Lewis等著, 计算理论基础, 清华大学出版社.

应用6：最大子段和

引例



Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7



Day	0	1	2	3	4
Price	10	11	7	10	6
Change		1	-4	3	-4

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

maximum subarray

最大子段和

- 给定整数序列 $a_1, a_2, \dots, a_n$ ，求形如 $\sum_{k=i}^j a_k$ 的子段和的最大值。规定子段和为负整数时，定义其最大子段和为0，即

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

- 例如， $(a_1, a_2, a_3, a_4, a_5, a_6) = (-2, 11, -4, 13, -5, -2)$
最大子段和为

$$\sum_{k=2}^4 a_k = 20$$

最大子段和

1 可以把所有的子段和计算出来，找到最小的

2 找到所有子段算法：

每个子段有一个起点 i 和一个终点 j

把起点位置 i 从左到右进行扫描

确定起点后，把终点位置 j ，左到右进行扫描，确定起点终点后，把这个子段中所元素相加 $(i, i+1, \dots, j)$,

最大子段和

```
int MaxSubSum2(int n, int a[], int &besti, int &bestj)
{ //数组a[]存储ai, 返回最大子段和, 保存起止位置到
  Besti,Bbestj中
    int sum=0;
    for(int i=1; i<=n; i++){
        int thissum=0;
        for(int j=i; j<=n; j++) {
            thissum += a[j];
            if(thissum>sum) {
                sum=thissum;
                besti=i; bestj=j;
            }
        }
    }
    return sum;
}
```

改进算法: $T(n)=O(n^2)$;

最大子段和: 分治算法

● 基本思想

将 $A[1..n]$ 分为 $a[1..n/2]$ 和 $a[n/2+1..n]$, 分别对两区段求最大子段和, 这时有三种情形:

Case 1: $a[1..n]$ 的最大子段和的子段落在 $a[1..n/2]$;

Case 2: $a[1..n]$ 的最大子段和的子段落在 $a[n/2..n]$;

Case 3: $a[1..n]$ 的最大子段和的子段跨在 $a[1..n/2]$ 和 $a[n/2..n]$ 之间;

最大子段和

- 对 Case 1 和 Case 2 可递归求解；

对 Case 3，可知 $a[n/2]$ 和 $a[n/2+1]$ 一定在最大和的子段中，因此

在 $a[1..n/2]$ 中计算：

$$S_1 = \max_{1 \leq i \leq n/2} \sum_{k=i}^{n/2} a_k$$

在 $a[n/2+1..n]$ 中计算：

$$S_2 = \max_{n/2+1 \leq i \leq n} \sum_{k=i}^n a_k$$

易知： $S_1 + S_2$ 是 Case 3 的最大值

```
int MaxSubSum3(int a[], int left, int right)
{ //返回最大子段和
    int sum=0;
    if(left==right)
        sum=a[left]>0?a[left]:0;
    else {
        int center=(left+right)/2;
        int leftsum=
            MaxSubSum3(a, left,center);
        int rightsum=
            MaxSubSum3(a, center+1, right);
        int s1=0; int leftmidsum=0;
        for(int i=center; i>=left; i--) {
            leftmidsum += a[i];
            if(leftmidsum>s1) s1=leftmidsum;
        }
    }
}
```

最大子段和

```
int s2=0; int rightmidsum=0;
for(int i=center+1; i<=right; i++) {
    rightminsum += a[i];
    if(rightmidsum>s2)
        s2=rightmidsum;
}
int sum=s1+s2;
if(sum<leftsum) sum=leftsum;
if(sum<rightsum) sum=rightsum;
} //end if
return sum;
} //end
```

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$
$$\Rightarrow T(n) = O(n \log n)$$

应用7: Strassen矩阵乘法

$$C=AB=(c_{ij})_{n \times n}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad i, j = 1, 2, \dots, n$$

求 $C=AB$ 即对 n^2 个元素 c_{ij} 进行计算，故要作 n^3 次乘法。相当时间内没有人怀疑过是否可以用少于 n^3 次乘法来完成。其实不然，先以 $n=2$ 的矩阵乘积为例。

对于矩阵

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

应用6: Strassen矩阵乘法

则有

$$\begin{aligned} C = AB &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \\ &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \\ &= \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \end{aligned}$$

共需作8次乘法。

应用6： Strassen矩阵乘法

- 计算2个 n 阶方阵的乘积转化为计算8个 $n/2$ 阶方阵的乘积和4个 $n/2$ 阶方阵的加法。
- 2个 $n/2 \times n/2$ 阶方阵的加法可以在 $O(n^2)$ 时间内完成。因此 $T(n)$ 满足下式，求得

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(\frac{n}{2}) + O(n^2) & n > 2 \end{cases}$$

■ $T(n) = O(n^3)$.

问题

问题：如何将乘法次数减为少于8次？

Strassen提出的算法如下：

$$\begin{aligned}\text{令} \quad & P = (a_{11} + a_{22})(b_{11} + b_{22}) \\ & Q = (a_{21} + a_{22})b_{11} \\ & R = a_{11}(b_{12} - b_{22}) \\ & S = a_{22}(b_{21} - b_{11}) \\ & T = (a_{11} + a_{12})b_{22} \\ & U = (a_{21} - a_{11})(b_{11} + b_{12}) \\ & V = (a_{12} - a_{22})(b_{21} + b_{22})\end{aligned}$$

则

$$\begin{aligned}c_{11} &= P + S - T + V \\ c_{12} &= R + T \\ c_{21} &= Q + S \\ c_{22} &= P + R - Q + U\end{aligned}$$

乘法次数从8次减为7次。上述方法可以推广到一般的矩阵乘积的情形。

应用6: Strassen矩阵乘法

- 设A和B是阶为 2^n 的方阵，只要将 $a_{11}, a_{12}, a_{21}, a_{22}$ 看作是 2^n 阶的子矩阵，结论仍然是成立的。

应用6: Strassen矩阵乘法

• 可以对Strassen算法的复杂性进行分析得到。

算法的所需的计算时间 $T(n)$ 满足递归方程:

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(\frac{n}{2}) + O(n^2) & n > 2 \end{cases}$$

Strassen矩阵乘法算法时间复杂度 **$O(n^{2.81})$**

Strassen矩阵乘法

- 有人提出了计算 2×2 阶矩阵乘法的36种不同方法，都要做7次乘法
- Hopcroft和Kerr(1971)已经证明，7次乘法是必要的。
- 应当研究 3×3 或 5×5 矩阵的更好算法；
- 目前最好的计算时间上界是 $O(n^{2.367})$ ，而最好下界仍是 $\Omega(n^2)$ 。

应用8：最接近点对问题

最接近点对问题

- 问题描述：给定平面上 n 个点，找其中的一对点，使得在 n 个点所组成的所有点对中，该点对间的距离最小。
- 说明：
 - 严格来讲，最接近点对可能多于一对，为简便起见，我们只找其中的一对作为问题的解。
 - 一个简单的做法是将每一个点与其他 $n-1$ 个点的距离算出，找出最小距离的点对即可。该方法的时间复杂性是 $T(n)=n(n-1)/2+n=O(n^2)$ ，效率较低。
 - 已经证明，该算法的计算时间下界是 $\Omega(n\log n)$ 。

最接近点对问题

- 输入: 平面上点集 $P = \{p_1, p_2, \dots, p_n\}$
- 输出: (s, t) 使得
$$d(p_s, p_t) = \min \{ d(u, v) \mid u \neq v \in P \}$$
其中设 $u = (x_1, y_1)$, $v = (x_2, y_2)$,

$$d(u, v) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

算法设计分析过程:

直接法--一维--排序--分治--二维--改进--改进

最近点对-逐对求距离

- 输入: 平面上点集 $P = \{p_1, p_2, \dots, p_n\}$
- 输出: (s, t) 使得 $d(p_s, p_t)$ 是最小点间距

$O(1)$ 1. 初始化: $\min = d(p_1, p_2); s=1; t=2;$

2. 对 $i = 1 : n-1$

3. 对 $j = i+1 : n$

$O(n^2)$

$O(1)$

4. 若 $\min > d(p_i, p_j),$

5. 则 $s = i; t = j; \min = d(p_i, p_j)$

6. 输出 (s, t)

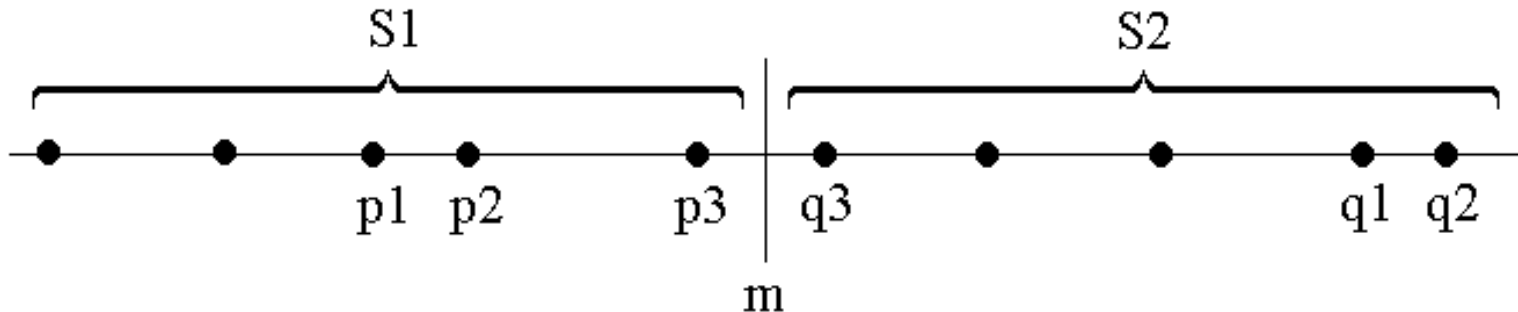
总时间 $O(C(n, 2)) = O(n^2)$

一维空间中的情形

- 为了使问题易于理解和分析，先来考虑一维的情形。此时， S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \dots, x_n$ 。最接近点对即为这 n 个实数中相差最小的2个实数。
- 一个简单的办法是先把 $x_1, x_2, \dots, x_n$ 排好序，再进行一次线性扫描就可以找出最接近点对， $T(n)=O(n\log n)$ 。然而这种方法无法推广到二维情形。

一维空间中的情形

- 假设我们用x轴上某个点 m 将 S 划分为2个子集 S_1 和 S_2 ，基于平衡子问题的思想，用 S 中各点坐标的中位数来作分割点。
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ， S 中的最接近点对或者是 $\{p_1, p_2\}$ ，或者是 $\{q_1, q_2\}$ ，或者是某个 $\{p_3, q_3\}$ ，其中 $p_3 \in S_1$ 且 $q_3 \in S_2$ 。
- 能否在线性时间内找到 p_3, q_3 ？



算法描述及复杂性

- 如果 S 的最接近点对是 $\{p_3, q_3\}$ ，即 $|p_3 - q_3| < d$ ，则 p_3 和 q_3 两者与 m 的距离不超过 d ，即 $p_3 \in (m-d, m]$ ， $q_3 \in (m, m+d]$ 。
- 由于在 S_1 中，每个长度为 d 的半闭区间至多包含一个点（否则必有两点距离小于 d ），并且 m 是 S_1 和 S_2 的分割点，因此 $(m-d, m]$ 中至多包含 S 中的一个点。由图可以看出，如果 $(m-d, m]$ 中有 S 中的点，则此点就是 S_1 中最大点。

算法描述及复杂性

- 问题： 请问能否在线性时间内找到 p_3 和 q_3 ？
- 因此，我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点，即 p_3 和 q_3 。从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为 S 的解。

算法描述及复杂性

- 分割点 m 的选取不当，会造成 $|S_i|=1$ ， $|S_j|=n-1(i+j=1)$ 的情形，使得 $T(n) = T(n-1) + O(n) = O(n^2)$ 。
- 这种情形可以通过“平衡子问题”方法加以解决：选取各点坐标的中位数作分割点。

最近点对--一维方法一

排序再逐个计算距离:

$O(n \log n)$ 1. 排序: $p_{i_1} \leq p_{i_2} \leq \dots \leq p_{i_n}$.

2. 初始化: $\min = d(p_{i_1}, p_{i_2}); s = i_1; t = i_2;$

3. 对 $k = 2 : n-1$

$O(n)$ {
 $O(1)$ {

4. 若 $\min > d(p_{i_k}, p_{i_{k+1}})$

5. 则 $s = i_k; t = i_{k+1}; \min = d(p_{i_k}, p_{i_{k+1}})$

● 总时间 $O(n \log n)$

● 不能推广到二维

最近点对--一维分治

问题1: 设点集合为 S , 如何分成两个部分 S_L 和 S_R ?

- 取中点 $m = (\min S + \max S)/2$ 划分, 可能不平衡
- 取中位数划分(解决了平衡问题)

问题2: 如何合并?

- 最小距离 = $\min \{ d_L, d_R, \min S_R - \max S_L \}$

$O(n)$ 1. 分: 取 S 中位数, 划分为 $S_L < S_R$.

$2T(n/2)$ 2. 治: 递归求 $S_L(S_R)$ 的最近点对距离 $d_L(d_R)$

$O(n)$ 3. 合: 取 S_L 最大点 p , S_R 最小点 q

$O(1)$ 4. $\delta = \min \{ d_L, d_R, \mathbf{q-p} \}$

$$T(n) = \begin{cases} O(1) & n \leq 3 \\ 2T(n/2) + O(n) & n > 3 \end{cases} = O(n \log n)$$

最近点对—二维分治尝试

设点集合为 S ,

1. 分: 取 S 横坐标中位数 mid , 划分为 $S_L <_x S_R$.

2. 治: 递归求 $S_L(S_R)$ 的最近点对距离 $d_L(d_R)$

3. 合: $d = \min \{ d_L, d_R \}$

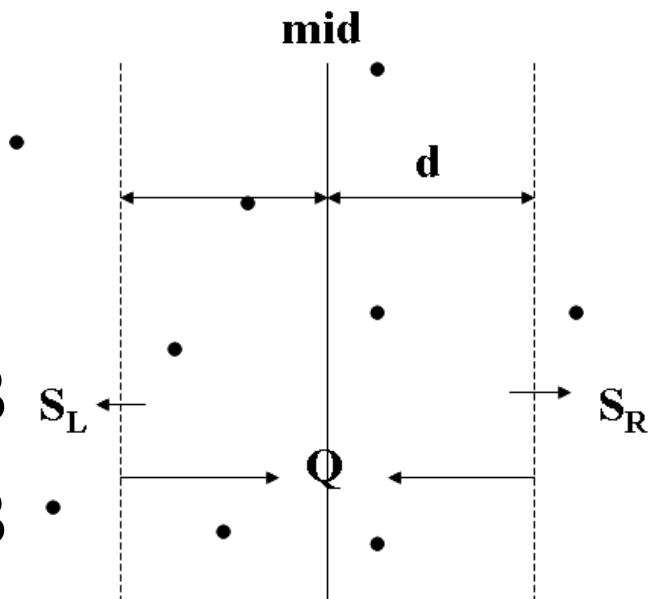
4. 取 $Q = \{ p \in S \mid |x(p) - \text{mid}| < d \}$

5. 逐对求 Q 中最近点对的距离 d .

分 $O(n)$, 治 $2T(n/2)$, 合 $O(n^2)$

根据分治主定理 $T(n) = O(n^2)$

$$T(n) = \begin{cases} O(1) & n \leq 3 \\ 2T(n/2) + O(n^2) & n > 3 \end{cases}$$



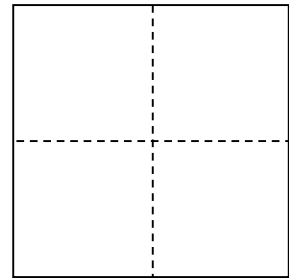
算法描述及复杂性

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

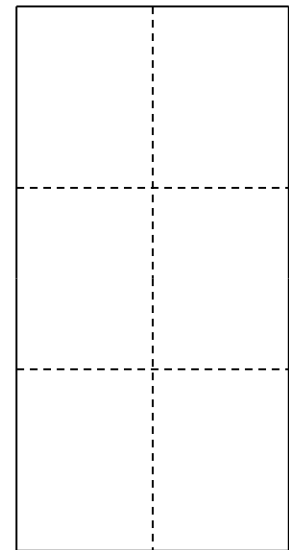
- 复杂性分析：
 - **$T(n) = O(n \log n)$**
 - 该算法可推广到二维的情形中去。

鸽巢(抽屉)原理的简单应用

任取一个 $d \times d$ 正方形内的点集 A ,
若 A 中任意两点距离都 $\geq d$, 则 A 中点数 ≤ 4 .



任取一个 $d \times 2d$ 矩形内点集 A ,
若 A 中任意两点距离都 $\geq d$, 则 A 中点数 ≤ 6 .



$$\sqrt{\left(\frac{d}{2}\right)^2 + \left(\frac{2d}{3}\right)^2} = \frac{5d}{6}$$

二维空间的最接近点对问题

- 下面来考虑二维的情形。
 - 选取一垂直线 $l:x=m$ 来作为分割直线。其中 m 为 S 中各点 x 坐标的中位数。由此将 S 分割为 S_1 和 S_2 。
 - 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2 ，并设 $d=\min\{d_1, d_2\}$ 。若 S 的最接近点对 (p,q) 之间的距离小于 d ，则 $p \in P_1$ 且 $q \in P_2$ ，如图2-9所示。

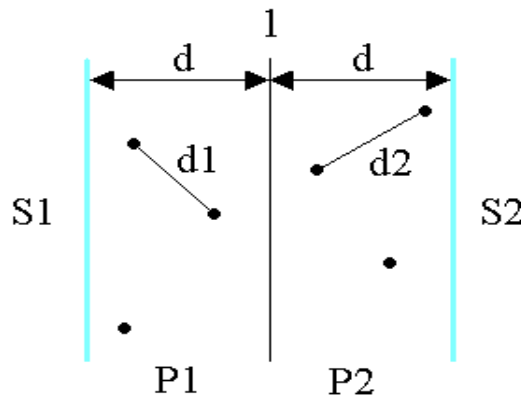


图2-9距离直线 l 小于 d 的所有点

二维空间的最接近点对问题

- 能否在线性时间内找到 p, q ?
 - 考虑 P_1 中任意一点 p ，它若与 P_2 中的点 q 构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的 P_2 中的点一定落在一个 $d \times 2d$ 的矩形 R 中，如图2-10所示。
 - 由 d 的意义可知， P_2 中任何2个 S 中的点的距离都不小于 d 。由此可以推出矩形 R 中最多只有6个 S 中的点。

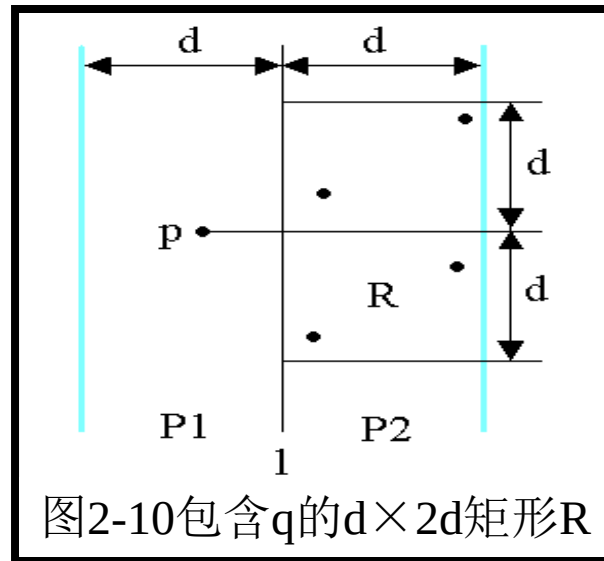


图2-10包含 q 的 $d \times 2d$ 矩形 R

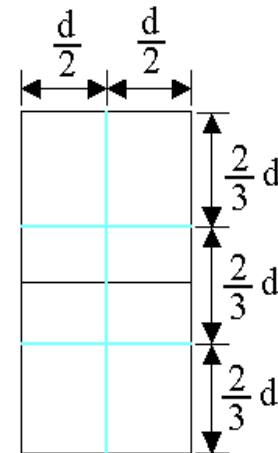
R中至多包含6个S中的点的证明

- 证明:

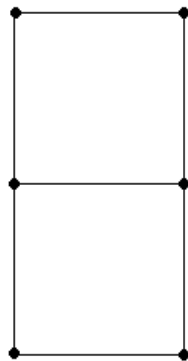
- 将矩形R的长为 $2d$ 的边3等分, 将它的长为 d 的边2等分, 由此导出6个 $(d/2) \times (2d/3)$ 的矩形 (如图(a)所示)。
- 若矩形R中有多于6个S中的点, 则由鸽舍原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上S中的点。
- 设 u, v 是位于同一小矩形中的2个点, 则

$$\begin{aligned} & (x(u) - x(v))^2 + (y(u) - y(v))^2 \\ \leq & (d/2)^2 + (2d/3)^2 = \frac{25}{36} d^2 \end{aligned}$$

- 因此, $\text{distance}(u, v) < d$ 。这与 d 的意义相矛盾。也
形R中最多有6个S中的点。



(a)



(b)

- 极端情形: 图(b)是矩形R中恰有6个S中的点的极端情形。

说明

- 因此，在分治法的合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者。
- 为了确切地知道要检查哪6个点，可以将 p 和 $P2$ 中所有 $S2$ 的点投影到垂直线 l 上。由于能与 p 点一起构成最接近点对候选者的 $S2$ 中点一定在矩形 R 中，所以它们在直线 l 上的投影点距 p 在 l 上投影点的距离小于 d 。由上面的分析可知，这种投影点最多只有6个。
- 因此，若将 $P1$ 和 $P2$ 中所有 S 中点按其 y 坐标排好序，则对 $P1$ 中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选者。对 $P1$ 中每一点最多只要检查 $P2$ 中排好序的相继6个点。

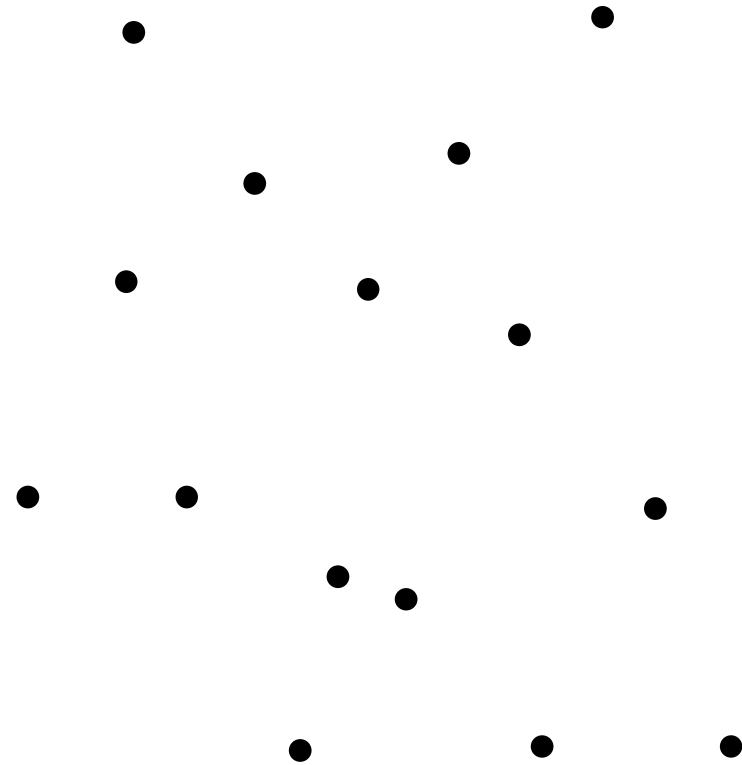
算法图示--初始

S

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

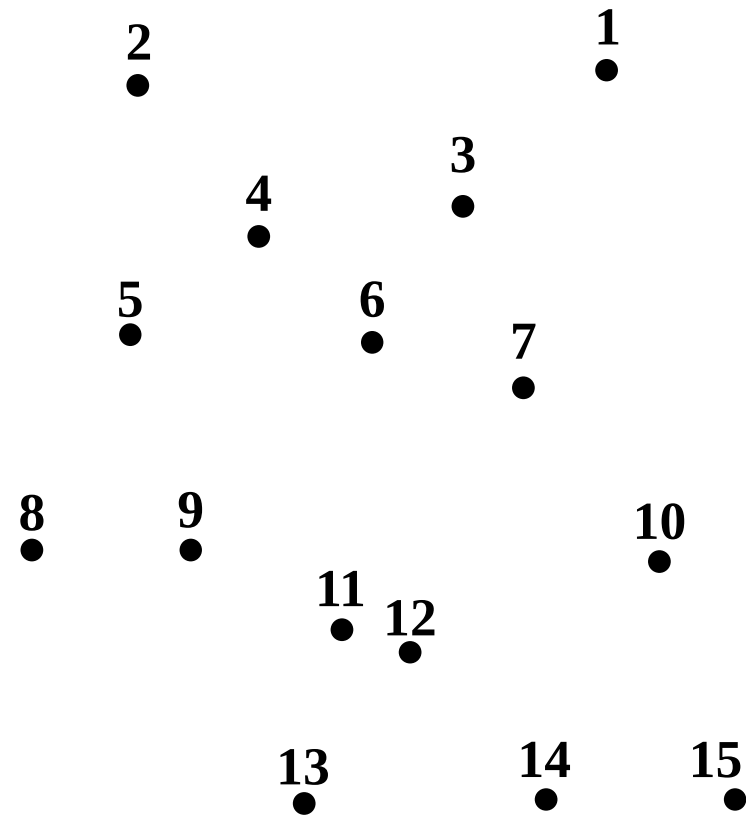


算法图示--预处理

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

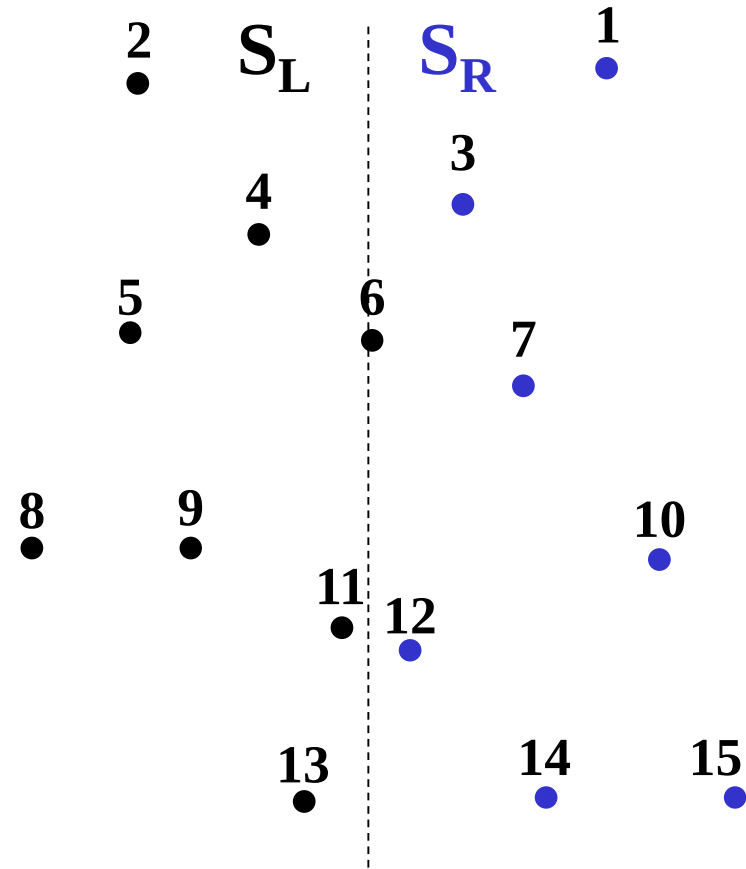


算法图示--分

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

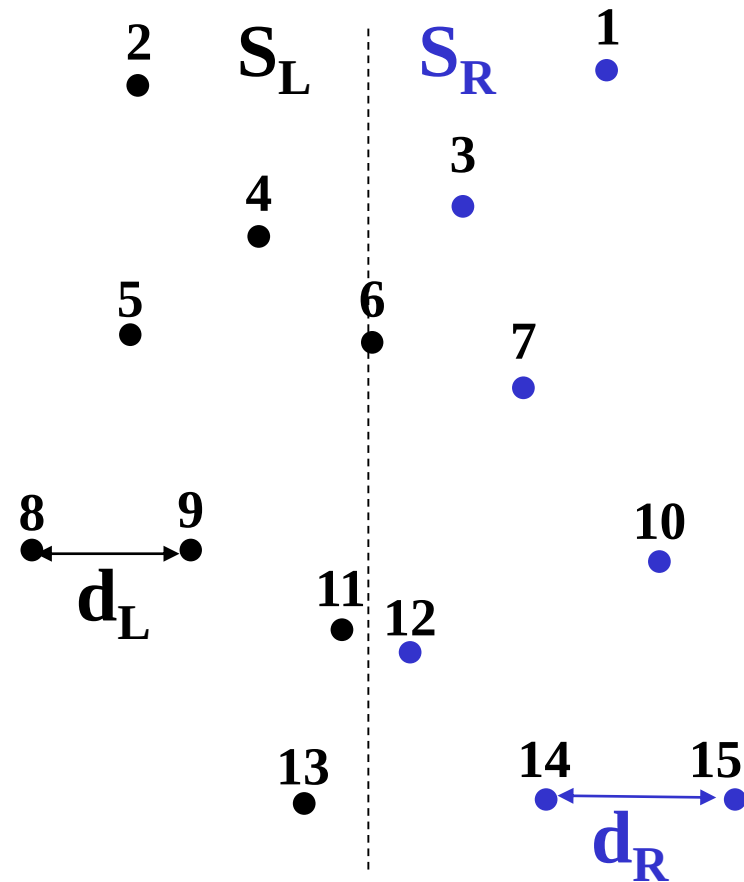


算法图示--治

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

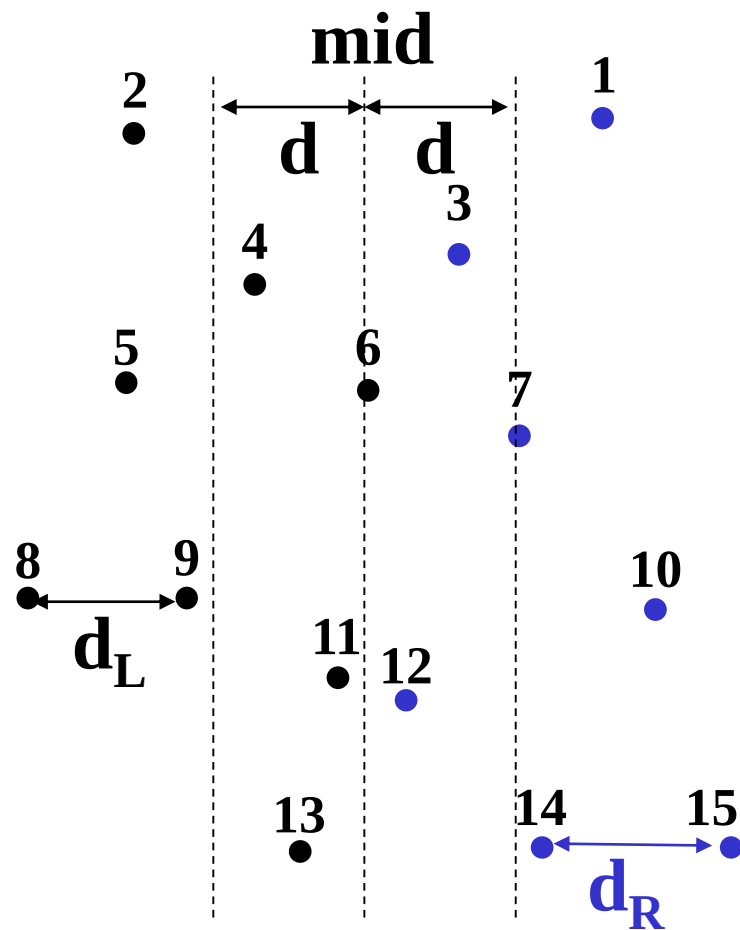


算法图示--合3

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

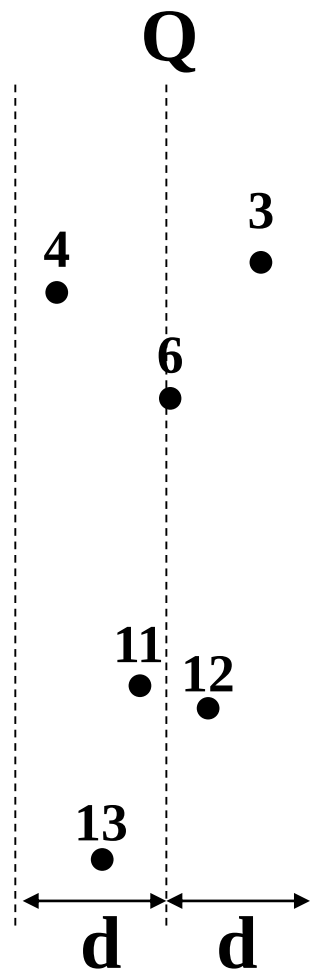


算法图示--合4

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

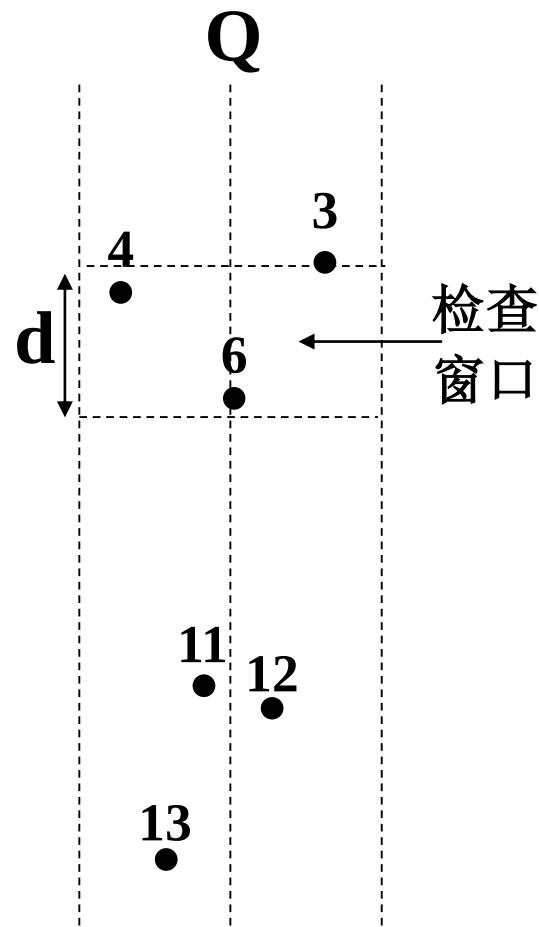


算法图示--合67:p₃

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离



算法图示--合67:p₄

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .

2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$

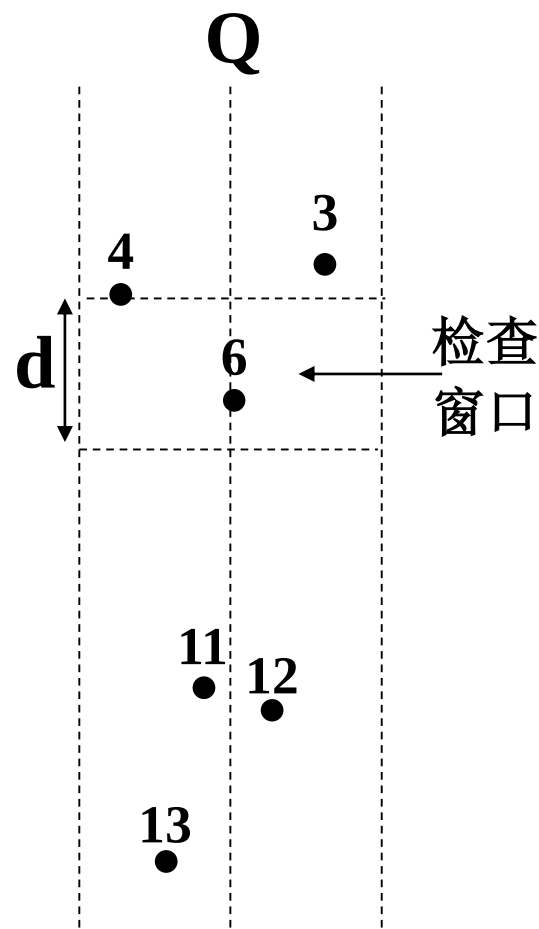
3. 合: $d = \min \{ d_L, d_R \}$

4. 由 S_L, S_R 按纵坐标大小归并得 Q

5. 对Q中每个点p,

6. 检查窗口 $R(p, d)$

7. 更新最短距离



算法图示--合67:p₆

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .

2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$

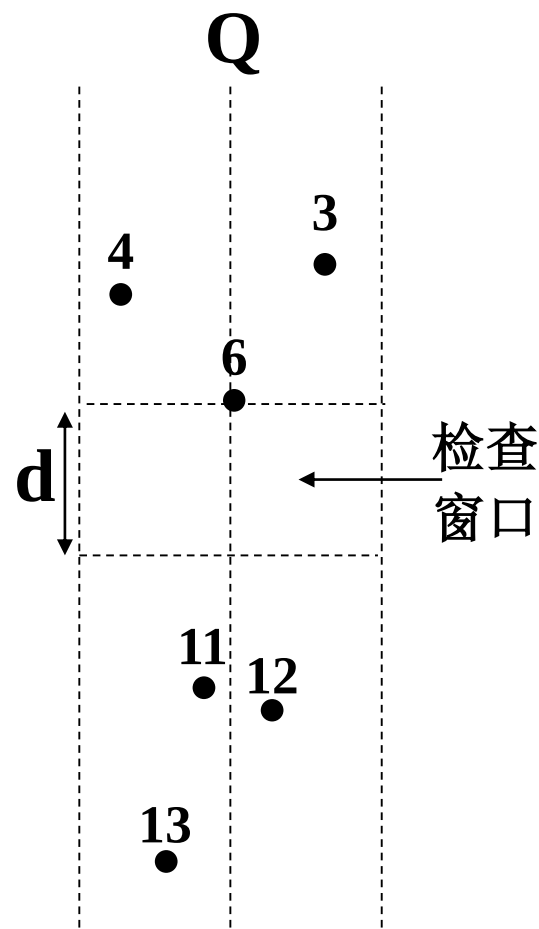
3. 合: $d = \min \{ d_L, d_R \}$

4. 由 S_L, S_R 按纵坐标大小归并得 Q

5. 对Q中每个点p,

6. 检查窗口 $R(p, d)$

7. 更新最短距离

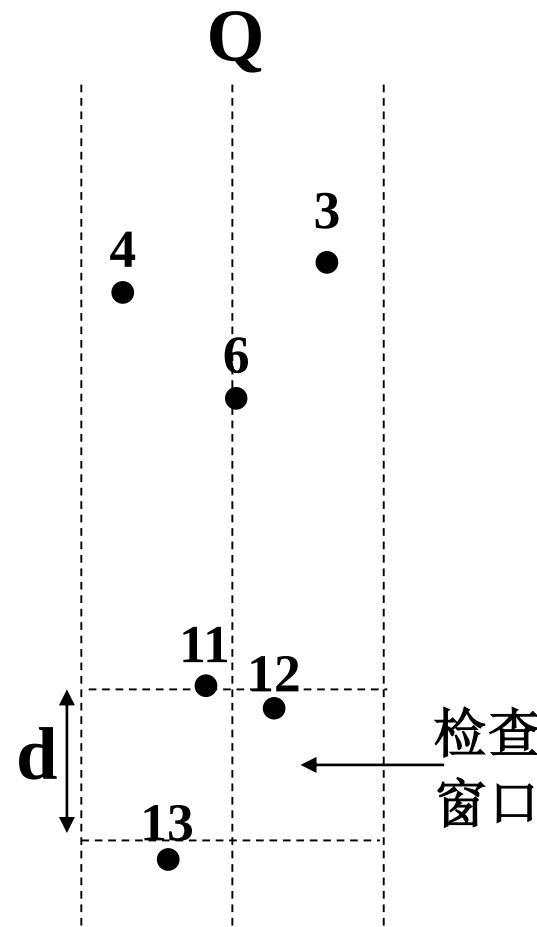


算法图示--合6:p₁₁

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. **检查窗口R(p,d)**
7. 更新最短距离

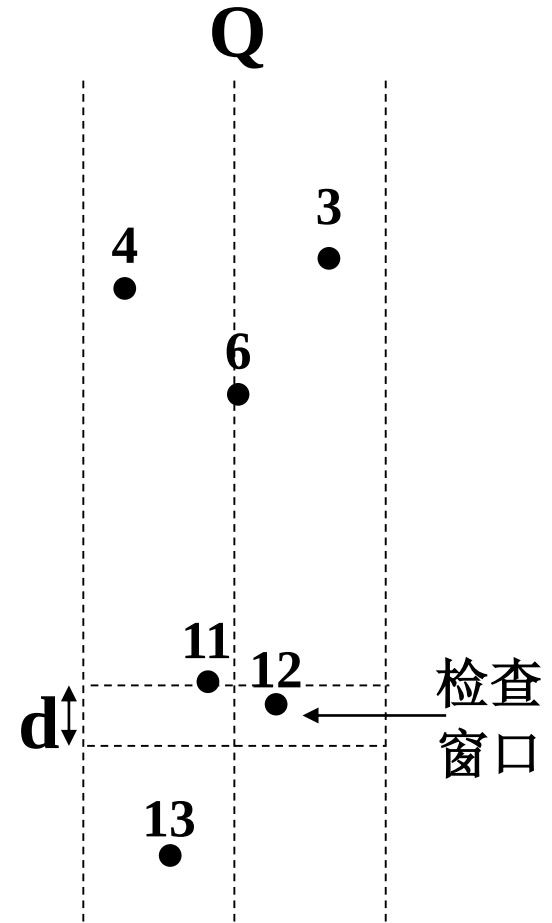


算法图示--合7:p₁₁

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. 检查窗口 $R(p, d)$
7. 更新最短距离

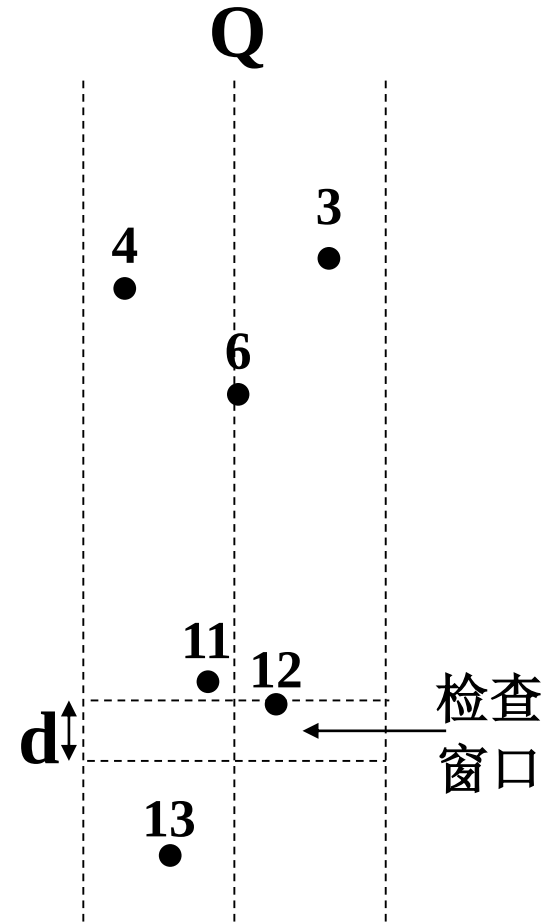


算法图示--合67:p₁₂

设有平面点集S

按y坐标递减(预处理)

1. 分: 取S横坐标中位数mid, 划分 S_L, S_R .
2. 治: 递归求 $S_L(S_R)$ 最近点对距离 $d_L(d_R)$
3. 合: $d = \min \{ d_L, d_R \}$
4. 由 S_L, S_R 按纵坐标大小归并得 Q
5. 对Q中每个点p,
6. **检查窗口R(p,d)**
7. **更新最短距离**



最近点对程序-定义

```
class PointX
{ public:
    int operator<=(PointX a) const
    {return(x<=a.x);}
private:
    int ID; //点编号
    float x,y;//点坐标
};

class PointY
{ public:
    int operator<=(PointX a) const
    {return(y<=a.y);}
private:
    int p; //同一点在数组X中的编号
    float x,y;//点坐标
};
```

```
template<class Type>
inline float distance(const Type& u,
                      const Type& v)
{
    float dx = u.x-v.x;
    float dy = u.y-v.y;
    return sqrt(dx*dx+dy*dy);
}
```

最近点对程序-预排序

```
bool Cpair2(PointX X[], int n, PointX& a, PointX& b, float& d)
{
    if(n<2)return false;
    MergeSort(X,n);           // X按横坐标排序
    PointY *Y = new PointY [n];
    for(int i = 0; i < n; i++) //将数组X中的点复制到数组Y中
    {
        Y[i].p = i;
        Y[i].x = X[i].x;
        Y[i].y = Y[i].y;
    }
    MergeSort(Y,n);           //Y按纵坐标排序
    PointY *Z = new PointY [n];
    closest(X,Y,Z,0,n-1,a,b,d); //求最近点对
    delete [] Y;
    delete [] Z;
    return true;
}
```

最近点对程序-输入

```
int main()
{
    int n;
    scanf("%d",&n);
    PointX *X = new PointX [n];
    float xx,yy;
    for(int i = 0; i < n; i++) //输入数组X
    {
        scanf("%f %f",&xx,&yy);
        X[i].ID = i; X[i].x = xx; X[i].y = yy;
    }
    PointX& a; PointX& b; float& d;
    Cpair2(X, n, a, b, d);
    printf("%d, %d, %.2f\n",a,b,d); //输出a,b,d.
}
```

最近点对程序

```
void closest(PointX X[], PointY Y[], PointY Z[], int l,
            int r, PointX& a, PointX& b, float& d)
{ if( r - l <= 2) {直接计算; return;}           //2点和3点的情形
  int m = ( l + r ) / 2; int f = l, g = m + 1; //多于3点的情形, 用分治法
  for( int i = l; i <= r; i++) if( Y[i].p > m ) Z[g++] = Y[i]; else Z[f++] = Y[i]; //分
  closest(X,Z,Y,l,m,a,b,d);                     //治: 左边
  float dr; PointX ar, br; closest(X,Z,Y,m+1,r,ar,br,dr); //治: 右边
  if( dr < d ) { a = ar; b = br; d = dr;}         //合: d
  Merge(Z,Y,l,m,r);                             //Z的两个有序段合并到数组Y
  int k = l; for( int i = l; i <= r; i++ ) //合: 从Y中取d矩形条内的点置于Z中
    if( fabs( X[m].x - Y[i].x ) < d ) Z[k++] = Y[i];
  for( int i = 1; i < k; i++)                     //合: 对d矩形条中的每点(Z[l:k-1])
  { for(int j = i+1; j < k && Z[j].y - Z[i].y < d; j++) //合: 检查R(p,d)中的点
    { float dp = distance( Z[i], Z[j]);
      if( dp < d){ d = dp; a = X[Z[i].p]; b = X[Z[j].p]; } //合: 更新最小距离
    }
  } }
```

算法描述及复杂性分析

- 算法描述:
 - **public static double CPair2(S)**
 - {
 - $n=|S|$;
 - **if** ($n < 2$) **return** ;
 - $m=S$ 中各点 x 间坐标的中位数;
 - 构造 $S1$ 和 $S2$;
 - $//S1=\{p \in S | x(p) \leq m\}$,
 - $S2=\{p \in S | x(p) > m\}$
 - $d1=cpair2(S1)$;
 - $d2=cpair2(S2)$;
 - $dm=\min(d1,d2)$;
 - 设 $P1$ 是 $S1$ 中距垂直分割线 l 的距离在 dm 之内的所有点组成的集合;
 - $P2$ 是 $S2$ 中距分割线 l 的距离在 dm 之内所有点组成的集合;
 - 将 $P1$ 和 $P2$ 中点依其 y 坐标值排序;
 - 并设 X 和 Y 是相应的已排好序的点列;

算法描述及复杂性分析

- 通过扫描X以及对于X中每个点检查Y中与其距离在 dm 之内的所有点(最多6个)可以完成合并;
- 当X中的扫描指针逐次向上移动时, Y中的扫描指针可在宽为 $2dm$ 的区间内移动;
 - 设 dl 是按这种扫描方式找到的点对间的最小距离;
 - $d = \min(dm, dl)$;
 - **return d;**
- }

- 复杂度分析:

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$\blacksquare T(n) = O(n \log n)$$

分治附录

附录：中位数原理

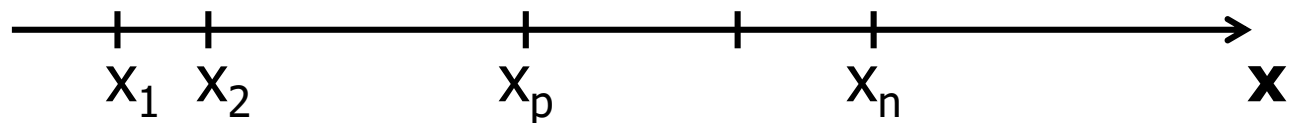
某公司有五个分公司依次设置在同一条铁路线的沿线A、B、C、D、E站。现在该公司希望在该铁路沿线设立一个仓库，要求该仓库离这五个站的火车行驶距离之和最小。如用数轴表示该铁路线，A、B、C、D、E各站的坐标依次为a、b、c、d、e（ $a < b < c < d < e$ ），则经过数学计算，该仓库大致应设置在坐标（1）处。

- (1) A. c B. $(a+b+c+d+e)/5$
C. $(a+2b+3c+2d+e)/9$
D. $(a+4b+6c+4d+e)/16$

附录：中位数原理

- 中位数原理

x 轴上有 n 个点，由左至右依次排列为

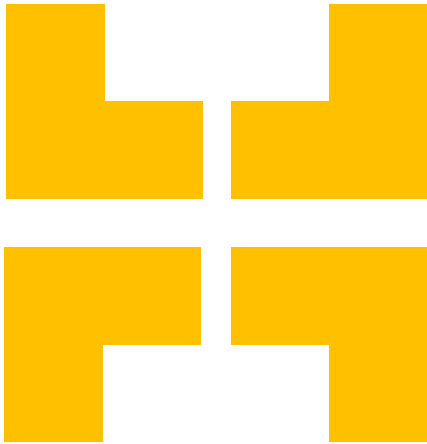


找一个点 x_p (不一定是 n 个点之一)，使 x_p 到各点距离和最小，解为：

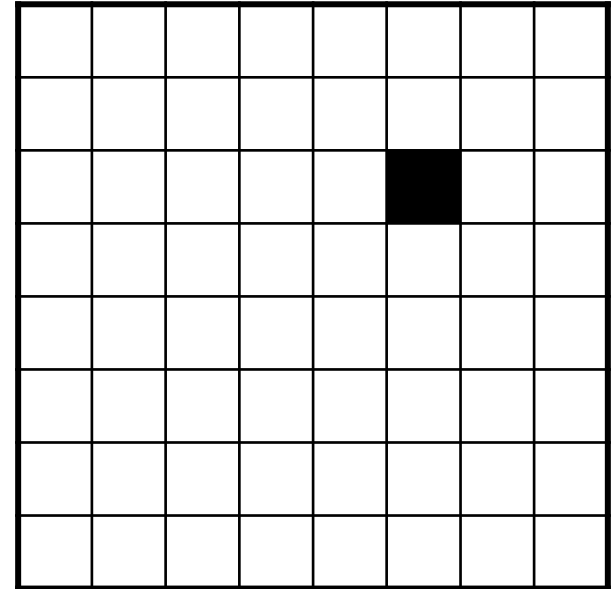
$$x_p = \begin{cases} x_{(n+1)/2} & \text{当 } n \text{ 为奇数时} \\ \text{中间两点的闭区间上} & \text{当 } n \text{ 为偶数时} \end{cases}$$

附录：棋盘覆盖

L型骨牌



$2^k \times 2^k$ 棋盘

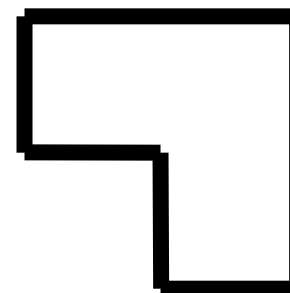
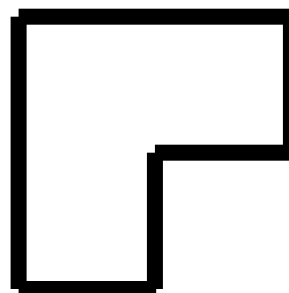
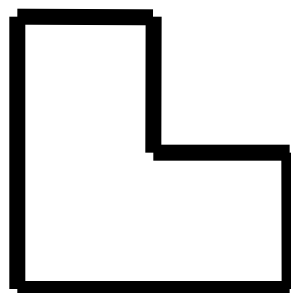
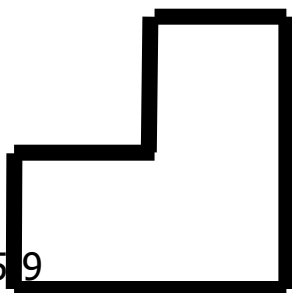
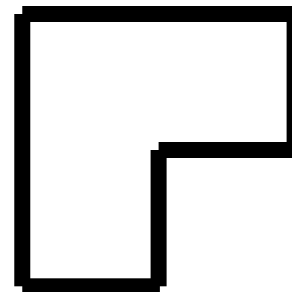
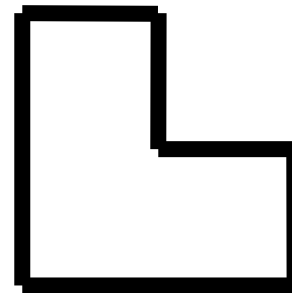
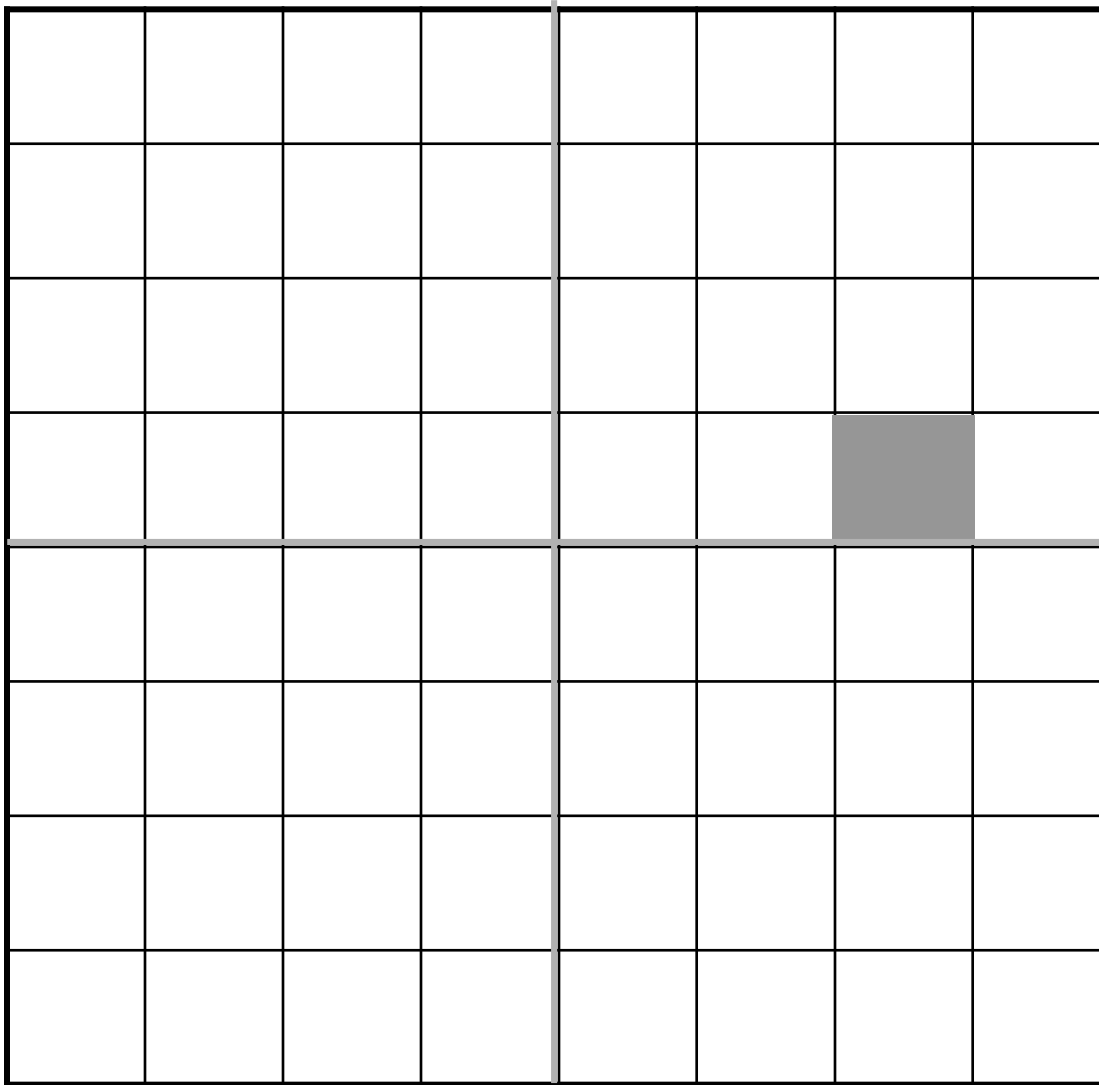


输入: k , 代表 $2^k \times 2^k$ 棋盘

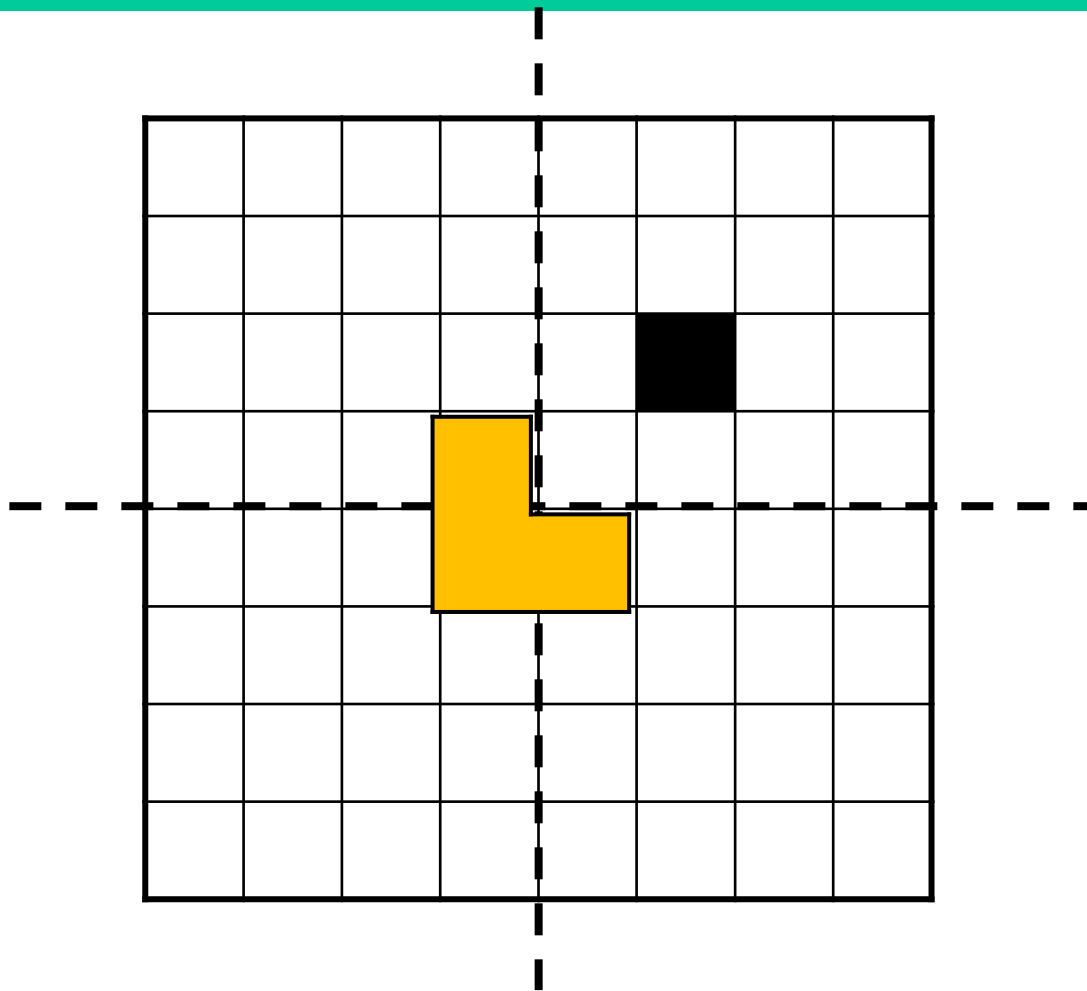
输出: 用 L 型骨牌覆盖棋盘的方案

说明: 有很多方案,
构造出一种方案即可

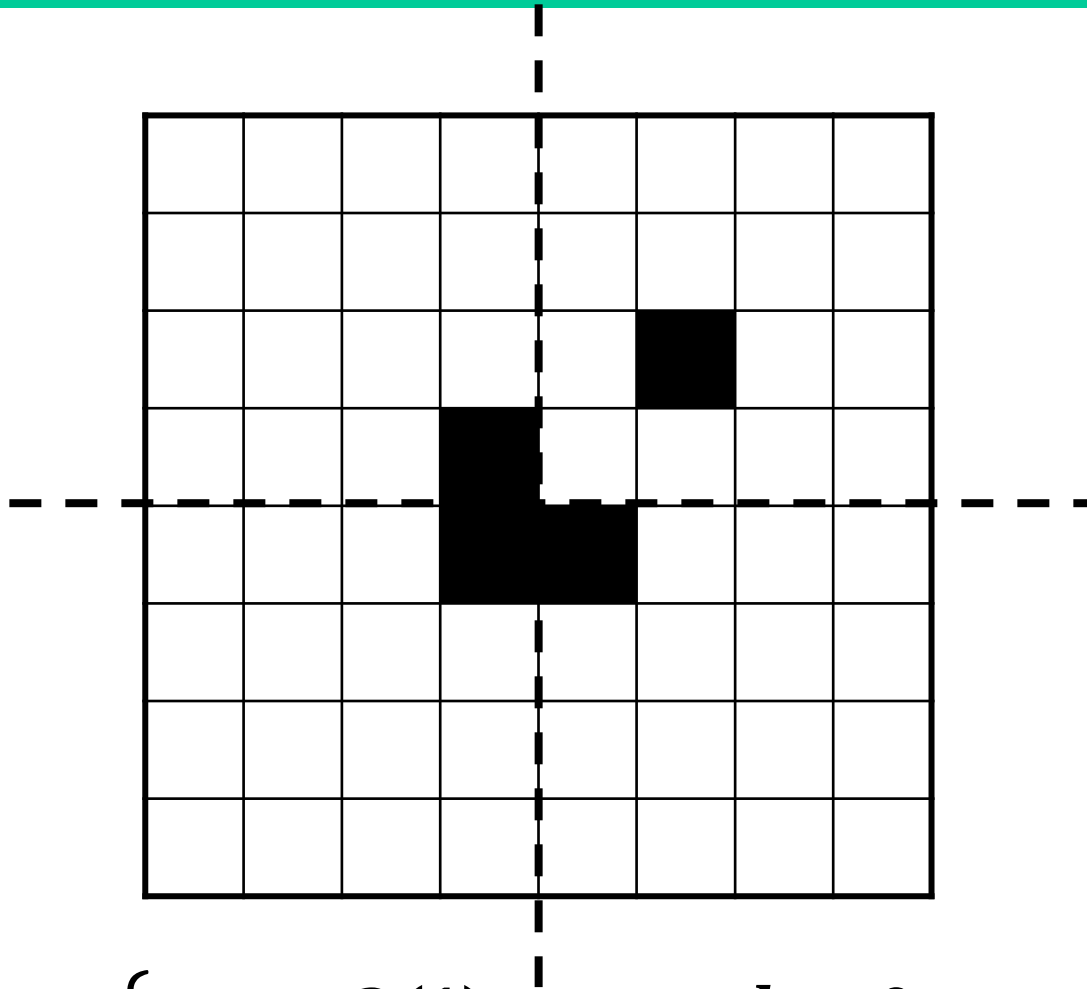
3	3	4	4	8	8	9	9
3	2	2	4	8	7	7	9
5	2	6	6	10		7	11
5	5	6	1	10	10	11	11
13	13	14	1	1	18	19	19
13	12	14	14	18	18	17	19
15	12	12	16	20	17	17	21
15	15	16	16	20	20	21	21



分治：递归构造



分治：递归构造



$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases} = O(4^k)$$

课堂练习-猜牌问题

甲手中有1张A，2张2，3张3，4张4，5张5，6张6，7张7，8张8，9张9共45张牌，现甲从中任取一张牌，然后乙开始提问来猜出这张牌。请给出乙提问的平均最少次数。

注：甲只能回答“是”或“否”。

作业

1-1 求下列函数的渐近表达式:

$3n^2+10n$; $n^2/10+2^n$; $2^{1+1/n}$; $\log n^3$; $10\log 3^n$.

1-2 试论 $O(1)$ 与 $O(2)$ 的区别.

作业

- 1-4 (1) 假设某算法在输入规模为 n 时的计算时间为 $T(n)=3 \times 2^n$. 在某台计算机上实现并完成该算法的时间为 t 秒. 现有另一台计算机, 其运行速度是第一台的64倍, 那么在这台新机器上用同一算法在 t 秒内能解输入规模为多大的问题?
- (2) 若上述算法的计算时间改进为 $T(n)=n^2$, 其余条件不变, 则在新机器上用 t 秒时间能解输入规模为多大的问题?
- (3) 若上述算法的计算时间改进为 $T(n)=8$, 其余条件不变, 那么在新机器上用 t 秒时间能解输入规模为多大的问题?

作业

2-8 设 n 个不同的整数排好序后存于 $T[1:n]$ 中. 若存在一个下标 i , $1 \leq i \leq n$, 使得 $T[i]=i$. 设计一个有效算法找到这个下标. 要求算法在最坏情况下的计算时间 $O(\log n)$.

2.9 设 $T[0:n-1]$ 是 n 个元素的数组. 对任一元素 x , 设 $S(x)=\{ i \mid T[i]=x \}$. 当 $|S(x)| > n/2$ 时, 称 x 为主元素. 设计一个线性时间算法, 确定 $T[0:n-1]$ 是否有一个主元素.